3

joining the dots

The Merge

five clean files finally speak to each other

Yesterday everything got clean. Today everything gets *connected*. By the end of the chapter, your five separate queries will resolve into two analysis-ready tables, and the year-on-year picture Priya needs will start to come together.

There are two ways to combine queries in Power Query, and learners confuse them all the time. **Append** stacks rows under each other (same columns, more rows). **Merge** brings columns alongside (more columns, usually the same rows). Get that distinction straight and the rest is click-by-click.

We'll also look at the six join types Power Query gives you, why merges beat VLOOKUP and XLOOKUP at their own game, and the one verification habit that catches almost every silent data error.

Append? Merge? Aren't they the same thing?

— you, on Wednesday morning, holding a third coffee

Different shapes. Different purposes. Once you see it, you can't unsee it.

we'll get there on page 3.

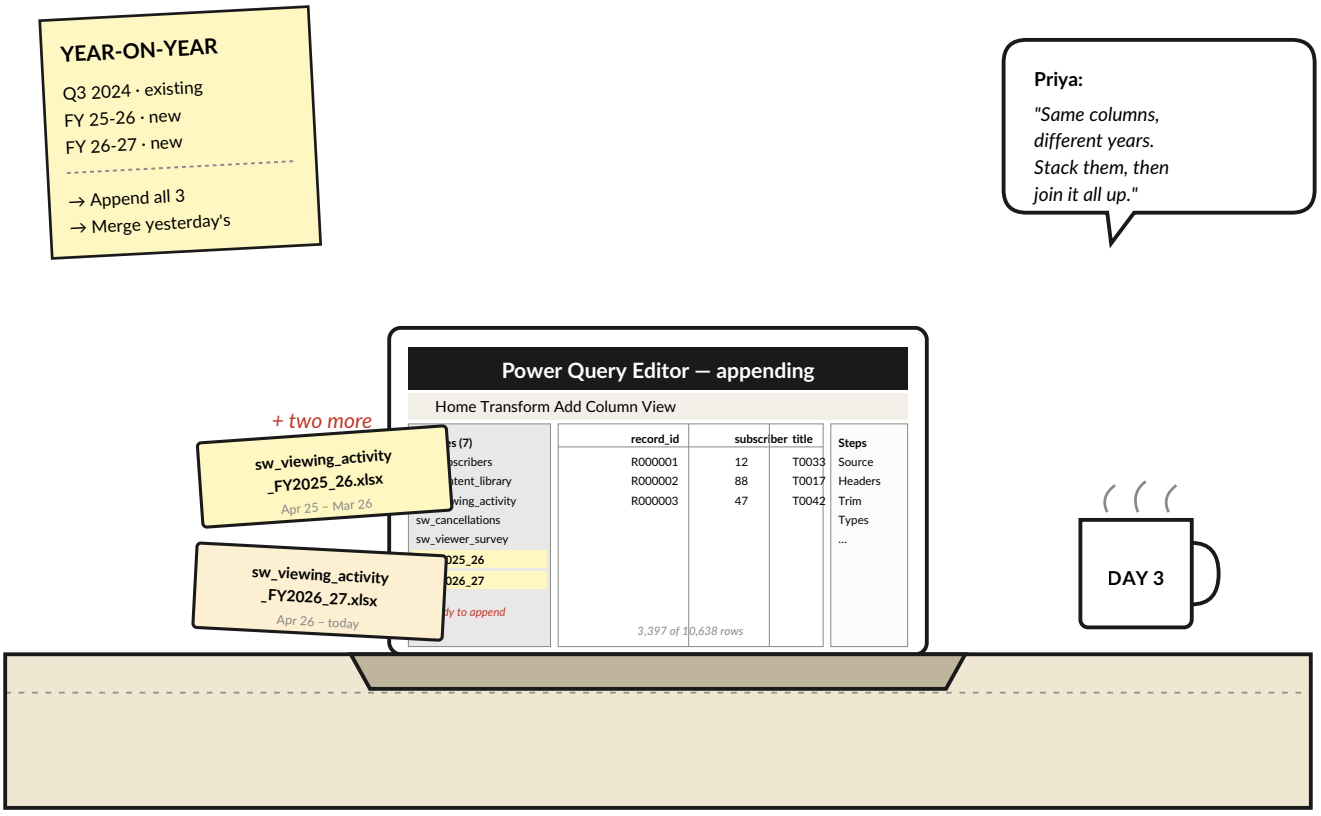
BRAIN POWER

You've cleaned five queries. Imagine merging two of them on `subscriber_id`. What's one thing that could still go wrong — even now, with clean keys? Jot one possibility down before turning the page.

turn over — and Priya drops two more files →

Two more files land

Priya pops over with two more Excel files for the year-on-year view. They have the same columns as yesterday's `sw_viewing_activity.xlsx` — just different time periods. They've been cleaned already. Good news: today is shorter than yesterday.



↑ Two new viewing files with the same shape as yesterday's. The right tool for this is **Append**, not Merge. We'll see why on the next page.

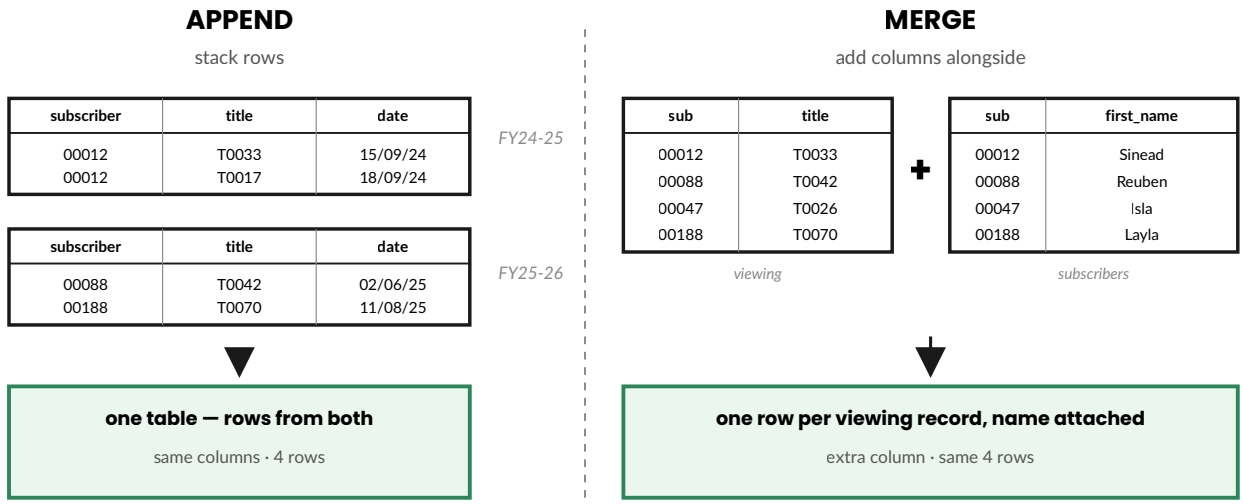
Right — same columns, more rows. Stack 'em.

Append vs Merge

One sentence each, and then the picture that makes it stick:

Append = stack rows from another query *below* the current one. Same columns, more rows. Use when you have files with the *same shape* (e.g. monthly sales, yearly viewing) and you want one combined table.

Merge = bring columns from another query *alongside*, matched on a key. More columns, usually the same rows. Use when you have *related* tables (e.g. viewing records and subscriber details) and you want one row per record with everything attached.



> *Append grows the rows. Merge grows the columns. That's the whole distinction.*

Append in practice

Three viewing activity files, same columns, different time periods. We want one combined query that covers everything from Q3 2024 right up to today.

The move

Home → Append Queries → Append Queries as New (the "as New" option creates a fresh query and leaves the three source queries untouched). In the dialog, pick **"Three or more tables"** and add each viewing query in turn:

- `sw_viewing_activity` (Q3 2024, 3,397 rows)
- `sw_viewing_activity_FY2025_26` (FY 25-26, ~6,120 rows)
- `sw_viewing_activity_FY2026_27` (FY 26-27 to date, ~1,121 rows)

Click OK. Power Query creates a new query — rename it `viewing_activity_all`.

[SCREENSHOT 1 — APPEND QUERIES DIALOG]

The Append Queries dialog with "Three or more tables" selected, the three viewing queries added on the right, "Append Queries as New" highlighted on the ribbon. Bonus: show the new query appearing in the Queries pane after OK.

Verify the count

Look at the bottom-left of the editor for the total row count. It should equal the sum of the source queries:

```
3,397 + 6,120 + 1,121 = 10,638 rows
```

Fewer than that → a query didn't append (column names probably don't match). More than that → something appended twice.

SHOW THIS OFF

Before and after the append, point at the row count in the bottom-left of the editor. Say: *"Three queries, 3,397 plus 6,120 plus 1,121. After append: 10,638. Numbers match — nothing dropped, nothing duplicated."* Verification narrated out loud is gold-band stuff.

Tour of the Merge dialog

Merge is more powerful than Append, and the dialog has more in it. Worth knowing where everything lives before you start clicking.

Home → Merge Queries → Merge Queries as New opens the dialog. Five things on it:

[SCREENSHOT 2 — MERGE QUERIES DIALOG (ANNOTATED)]

The full Merge dialog open. Top pane: left (primary) query with one column highlighted. Bottom pane: right (related) query dropdown selected with the matching column highlighted. Below: Join Kind dropdown. At the bottom of the dialog: the "Matching rows" indicator (e.g. "10,638 of 10,638 rows match"). Callouts to all five elements.

1. **Top pane (left query).** Your primary table — the one you're starting from. Click a column heading to choose the matching key.
2. **Bottom pane (right query).** Pick the query you want to merge in from the dropdown. Click the matching column heading.
3. **Join Kind dropdown.** Six options — see next page.
4. **"Use fuzzy matching" checkbox.** Leave unchecked for our work. Fuzzy matching is for when keys nearly match (e.g. "John Smith" vs "John Smyth"). Ours are now canonical, so we don't need it.
5. **Matching rows indicator.** The most useful number in the dialog. Tells you how many rows on the left have a match on the right, *before* you click OK. Always glance at this.

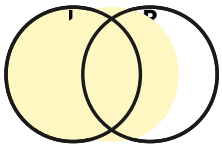
SHOW THIS OFF

Before you click OK on any merge, hover the cursor over the matching-rows indicator and say it out loud: *"10,638 of 10,638 rows match. No subscribers are about to silently drop out."* That one sentence shows the assessor you understand what could go wrong.

The six join types

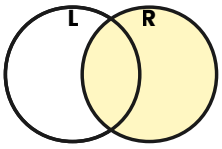
The Join Kind dropdown gives you six choices. The diagrams below tell you which rows end up in the result. **L** is your left (primary) query; **R** is the right (related) one. The shaded region is what survives the merge.

LEFT OUTER
your default



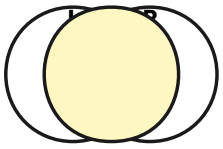
all of L, matches from R

RIGHT OUTER
rarely used



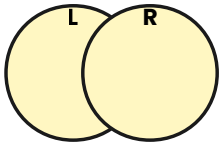
all of R, matches from L

INNER
strict — matches only



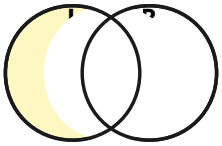
only rows that match both

FULL OUTER
keep everything



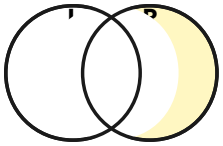
every row from both sides

LEFT ANTI
find the gaps in L



L rows with NO match in R

RIGHT ANTI
find the gaps in R



R rows with NO match in L

You'll spend most of your time on **Left Outer** (the default), reach for **Inner** when you need stricter matching, and pull out a **Left Anti** when you need to find subscribers (or rows) that don't appear in another table.

Joins by example

Left Outer — your default

You want to enrich one table with attributes from another, keeping every row from the original.

Example: Merge `viewing_activity_all` (left) with `sw_subscribers` (right) on `subscriber_id`. Every viewing record keeps its row; subscriber details get attached where they match. Subscribers without viewing activity simply don't appear (that's fine — they're not in our left table).

Inner — only matched rows

You want strict matches only; rows that can't match are dropped.

Example: Merge `sw_cancellations` (left) with `sw_viewer_survey` (right) on `subscriber_id`. Only cancelled subscribers *who also filled in a survey* come through. Useful when you specifically want intersection — e.g. "what reasons did the people who actually told us choose?"

Left Anti — find the gaps

You want to find rows that *don't* have a match in another table. The most under-used join, but brilliant for data-quality checks.

Examples:

- Merge `sw_subscribers` (left) with `viewing_activity_all` (right), Left Anti on `subscriber_id` → **subscribers who haven't watched anything**. Worth a chat with the marketing team.
- Merge `viewing_activity_all` (left) with `sw_subscribers` (right), Left Anti → **viewing records with no matching subscriber**. That should be zero rows. If it isn't, your canonical key has a gap.

ASSESSOR FAVOURITE

If the assessor asks *"how would you find subscribers who never watched anything?"* — Left Anti is the answer. Be ready with that one sentence. It's the kind of question that separates a pass from a distinction.

VLOOKUP, XLOOKUP, or Merge?

If you've been using VLOOKUP or XLOOKUP for years, Power Query Merge is going to feel like an upgrade — because that's exactly what it is. They solve similar problems; Merge just does it better.

Feature	VLOOKUP	XLOOKUP	PQ Merge
Bring one column over	✓	✓	✓
Bring *multiple* columns over in one move	✗	one per formula	✓
Survives column re-ordering	✗ (uses col index)	✓	✓
Recalcs on every workbook change	✓ (slow)	✓ (slow)	**only on Refresh**
Audit trail of what was done	✗	✗	✓ **(Applied Steps)**
Handles one-to-many cleanly	✗	✗	✓ **(fans out)**
Built-in "find non-matches"	workaround	workaround	✓ **(Left Anti)**
Live formula in a worksheet cell	✓	✓	— (lives in PQ)

The only thing VLOOKUP and XLOOKUP do that Merge doesn't: live in a single cell, recalculating instantly. If you need a one-off lookup in a tab someone shares with you over email, the formulas still win. For anything you're going to do *more than once* — anything with refresh, anything auditable, anything that handles real data volumes — Merge is the answer.

ASSESSOR FAVOURITE

If asked *"why not just use VLOOKUP?"*, your one-sentence answer: *"Because Merge is repeatable on refresh, auditable in Applied Steps, brings multiple columns over in one move, and handles non-matches as a feature rather than an error."*

Verifying — count the rows

Every merge can silently lose or duplicate rows. Every one. So *every* merge needs a count check. This is the habit that separates careful work from careless work.

Before you click OK

The dialog shows **matching rows** at the bottom (e.g. *"10,638 of 10,638 rows match"*). If that second number doesn't equal your left-query row count, the merge is going to drop something. Pause and investigate.

After the merge runs

Check the row count in the bottom-left of the editor. For a Left Outer:

- **Same row count as before:** good — one-to-one match, no fan-out.
- **More rows than before:** you've got *fan-out* — duplicate keys on the right side. Each match multiplies the row.
- **Fewer rows than before:** impossible in a Left Outer. If you see fewer, something else is wrong (you picked Inner by mistake?).

For an Inner join, fewer rows is *expected* — that's the whole point. But you still want to know how many you lost, and why.

[SCREENSHOT 3 — ROW COUNT BEFORE/AFTER]

Bottom-left of the Power Query Editor showing total row count, ideally captured twice — once before the merge step and once after — with the Applied Steps pane visible on the right showing the new "Merged Queries" step.

SHOW THIS OFF

Every merge you do, narrate the count: *"Before: 10,638 rows. After: 10,638. One-to-one match, no fan-out, no drops."* Don't skip this even if the numbers look obviously right — the act of saying it out loud is what the assessor is looking for.

Halfway House

Four scenarios — pick the right tool (Append, Left Outer, Inner, or Left Anti). Answers below the line; wrong answer = the page you should re-read.

1. You have three monthly sales files with the same columns. You want a single year-to-date table.

☐ **Append** ☐ **Left Outer** ☐ **Inner** ☐ **Left Anti**

2. You have an orders table and a products table. You want each order to also show the product name and category.

☐ **Append** ☐ **Left Outer** ☐ **Inner** ☐ **Left Anti**

3. You have a customers table and an orders table. You want only the customers who have placed at least one order.

☐ **Append** ☐ **Left Outer** ☐ **Inner** ☐ **Left Anti**

4. You have a customers table and an orders table. You want only the customers who have *never* placed an order — for a re-engagement campaign.

☐ **Append** ☐ **Left Outer** ☐ **Inner** ☐ **Left Anti**

Answers — check yourself

1. Append. Same columns, more rows. Always Append.

2. Left Outer. Keep every order, attach product details where they match.

3. Inner. Drops customers without a matching order. That's the filter.

4. Left Anti. Customers on the left, orders on the right, anti-join gives you customers who don't appear in orders.

The merge sequence

You've appended the three viewing files into `viewing_activity_all` and you have four other clean queries. Now we connect them. There's a sensible order, and we'll follow it.

The end state: **two analysis-ready tables**. One enriched activity table (every viewing record, every dimension attached) and one enriched subscribers table (every subscriber, with cancellation and survey context).

1

Merge `viewing_activity_all` ← `sw_subscribers`

Left Outer on `subscriber_id`. Brings region, plan_type, status, is_churned alongside every viewing record. Expand to bring the columns you want.

> *"Left Outer because every viewing record should keep its row, even if for some reason the subscriber isn't in the table. Verify: rows unchanged at 10,638."*

2

Merge result ← `sw_content_library`

Left Outer on `title_id`. Brings genre, content_type, release_year, age rating alongside each viewing record. Now each row knows *who* watched *what* as well as the title's attributes.

> *"Verify: rows still 10,638. No fan-out because every title_id has at most one matching row in the content library."*

3

Merge `sw_subscribers` ← `sw_cancellations`

Left Outer on `subscriber_id`. Brings cancel_date and stated_reason alongside subscribers. Active subscribers get nulls in these columns (which is correct — they haven't cancelled).

> *"Important: I de-duped the cancellations on Day 2, so there should be no fan-out. Verify."*

4

Merge result ← `sw_viewer_survey`

Left Outer on `subscriber_id`. Brings the satisfaction ratings and reason_code alongside subscribers. Subscribers who didn't fill in the survey get nulls.

> *"All four steps Left Outer because the subscribers table is the master — every subscriber stays, dimensions get attached where available."*

Once these four merges are done, load each enriched query to a sheet (Connection Only no longer needed for the final tables). You're ready for Day 4 — analysis.

SHARPEN YOUR PENCIL plan the merge — no software needed

You have three queries: `orders` (10,000 rows), `customers` (2,500 rows), `products` (340 rows). Your manager wants one table that shows every order with the customer's region and the product's category attached.

1. Which query is the left side of your first merge?

2. Which key column do you match on?

3. Which join kind?

4. After the merge runs, how many rows should the result have, and why?

Answers — check yourself

1. `orders`. It's the table you want every row preserved from — every order ends up with attributes attached, regardless of whether a particular customer or product has been ordered before.

2. `customer_id` for the customers merge; `product_id` for the products merge. Two separate merges, one per dimension.

3. Left Outer. Strict (Inner) would drop orders for customers or products you didn't have a match for — usually not what you want for an order-by-order analysis.

4. Still 10,000 rows — assuming no fan-out from duplicate keys in the dimension tables. If you suddenly have more rows than orders, you've got duplicate `customer_id` or `product_id` rows in those tables. De-dupe and re-merge.

There are no *Dumb* Questions

Q: *I tried to merge and got far more rows than I started with. What happened?*

A: You've got *fan-out* — duplicate keys in the right-hand table. Each duplicate creates an extra matching row. Either de-dupe the right table first (typical fix), or change your merge to use a more specific key. The verify-row-count habit catches this every time.

Q: *Why "Merge as New" rather than "Merge in place"?*

A: "As New" creates a new query and leaves the source query unchanged. Good for clarity and good for re-use — you keep the original query available for other merges. "In place" modifies the current query directly; useful when you're certain the merge is final, but easier to undo if you stick with "as New".

Q: *Should I merge before or after applying my Cleaning Sequence?*

A: After. Always. Merging dirty data multiplies the mess — and merging on a non-canonical key drops rows silently. Day 2 (clean each query) before Day 3 (merge) is the deliberate order.

Q: *If I append two queries and they have nearly matching column names — say `watch_date` in one and `Watch Date` in the other — what happens?*

A: Power Query treats them as *different* columns. You'll end up with both columns in the result, each null for the half of the rows where it doesn't exist. The fix: rename the columns to match exactly before appending. This is why Step 0 on Day 2 (rename early) matters.

Q: *How do I know which side should be left and which should be right?*

A: Left is the table you want every row of. Right is the table you want to enrich the left with. For "viewing records + subscriber attributes", viewing is left. For "subscribers + their viewing history", subscribers is left. If you're uncertain, ask: *"which table is my analysis row-by-row?"* — that's your left.

Q: *Can I undo a merge if I get the wrong join kind?*

A: Yes — click the gear icon next to the "Merged Queries" step in Applied Steps to reopen the dialog and change the join kind. The change re-applies immediately. You can also delete the step and start over.

> *Merge is a power tool. Count the rows before and after, every single time.*

Say it out loud

The assessor will absolutely ask you about your merging. Practise these three out loud now, as if they're across the table.

Q1 — Walk me through the difference between Append and Merge.

Aim for two clean sentences. *"Append stacks rows from another query below the current one — same columns, more rows. Merge brings columns from another query alongside, matched on a key — more columns, usually the same rows."* Then point at the year-on-year append you just did as the example.

Q2 — Why Left Outer for the StreamWave merges and not Inner?

The shape: state what each option does, name the risk, name the choice.

Try: *"Left Outer keeps every row from the left table even if there's no match — for viewing activity merging in subscriber details, that's what I want. Inner would silently drop any viewing record where the subscriber didn't match, and I'd never see it. Left Outer plus row-count verification gives me both safety and visibility."*

Q3 — How would you find subscribers who never watched anything?

One sentence. *"Left Anti join. Left side is the subscribers query, right side is the viewing activity, joined on subscriber_id. The result is just the subscribers with no match in viewing — the people who signed up but never watched."*

DISTINCTION TELL

For every merge, narrate **three things**: which side is left and why, which join kind you chose and what it does, and the row count before vs after. Three sentences. That's distinction-band evidence that you understand what a merge is actually doing.

Bullet Points — the chapter in 30 seconds

- **Append stacks rows. Merge adds columns.** Different operations, different purposes.
- **Six join types.** Left Outer is your default. Inner for strict matches. Left Anti to find gaps.
- **Power Query Merge beats VLOOKUP/XLOOKUP** on every dimension except being a single-cell formula.
- **Verify with row counts.** Every merge. Every time. Before and after.
- **Clean first, merge after.** The Day 2 → Day 3 order matters.

TEST DRIVE

Before closing the laptop, run this self-check. Tick each box only when it's true:

- ☐ Three viewing activity files appended into `viewing_activity_all` — row count = 10,638.
- ☐ `viewing_activity_all` merged with subscribers (Left Outer) then with content_library (Left Outer) — row count still 10,638.
- ☐ `sw_subscribers` merged with cancellations (Left Outer) then with viewer_survey (Left Outer).
- ☐ You can sketch the six join-type Venn diagrams from memory.
- ☐ You can give the one-sentence answer to *"why not just use VLOOKUP?"* without thinking.
- ☐ For every merge you ran today, you spoke the row count out loud at least once.

COMING UP — DAY 4

Reading the Data. Tomorrow the data finally tells you something. *The Descriptive Five* — count, mean, median, range, distribution — applied properly. *Group By* for summary tables. *SUM/COUNT/AVERAGE* as aggregations. The difference between counting rows and counting distinct values. And the first proper insights from the year-on-year view.

that's Day 3 done — your five files finally speak to each other.